Week 08: Normalization

Learning outcomes: CLO2, CLO3

Lesson

A **functional dependency** X → Y means that one value of X always gives the same value of Y. In our data, `province_pcode → province_name, area_km2` and `district_pcode → district_name, area_km2, province_pcode`.

The worked example joins the three tables into one flat report with one row per commune. That is easy to read, but each province's name and area are repeated on every commune row. If that flat table were the only copy of the data, it would suffer from anomalies:

- **update anomaly:** a province name stored in many rows must be changed in all of them;
- **insert anomaly:** a new province could not be added until it had a commune;
- **delete anomaly:** deleting a province's last commune would also lose the province.

1NF: each cell holds one value. **2NF:** the table is in 1NF and every non-key column depends on the whole key, not just part of it. This matters only for composite keys. For example, in an enrolment table with the key (`student_id`, `course_code`), a `course_name` column would break 2NF, because it depends on `course_code` alone. Our three tables each have a one-column key, so they cannot break 2NF. **3NF:** the table is in 2NF and no non-key column depends on another non-key column. In the flat report, `commune_pcode → district_pcode → province_name` is such a transitive dependency, so normalizing splits the report back into three tables. **BCNF:** for every functional dependency X → Y (where Y is not part of X), X must be a superkey, that is, a set of columns that contains a candidate key.

A denormalized table is fine as a read-only report, as long as you can rebuild it from the normalized tables.

Worked example

Run `example.sql` (see Run a SQL file (`setup/README.md`)) and read the output. Then open `work.sql`.

Tables for this week

Read from the `cambodia` database and make your changes in `student_work`. PK means primary key and FK means foreign key. Every column is `NOT NULL`. The sample rows are the first two rows of each table, sorted by code.

`cambodia.province`

Column	Type	Key	Meaning
`province_pcode`	`CHAR(4)`	PK	Province code
`province_name`	`VARCHAR(100)`		Province name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place

Sample rows:

province_pcode	province_name	area_km2
KH01	Banteay Meanchey	6145.9
KH02	Battambang	11869.0

cambodia.district

Column	Type	Key	Meaning
district_pcode	CHAR(6)	PK	District code
district_name	VARCHAR(100)		District name (English)
area_km2	DECIMAL(12,1)		Area in km², one decimal place
province_pcode	CHAR(4)	FK	Province code

Sample rows:

district_pcode	district_name	area_km2	province_pcode
KH0102	Mongkol Borei	807.2	KH01
KH0103	Phnum Srok	761.7	KH01

cambodia.commune

Column	Type	Key	Meaning
commune_pcode	CHAR(8)	PK	Commune code
commune_name	VARCHAR(100)		Commune name (English)
area_km2	DECIMAL(12,1)		Area in km², one decimal place
district_pcode	CHAR(6)	FK	District code

Sample rows:

commune_pcode	commune_name	area_km2	district_pcode
KH010201	Banteay Neang	52.0	KH0102
KH010202	Bat Trang	40.3	KH0102

Join keys: `district.province_pcode = province.province_pcode`; `commune.district_pcode = district.district_pcode`. Join on the codes, not the names.

Joined report columns: `commune_pcode`, `commune_name`, `commune_area_km2`, `district_pcode`, `district_name`, `district_area_km2`, `province_pcode`, `province_name`, `province_area_km2`. Each row is one commune. This is a query result, not a stored table.

Run `DESCRIBE cambodia.province;` to see a table definition in MySQL. The data guide (`data/README.md`) explains where the data comes from.

Lab tasks

1. Run the joined report from `example.sql` and save its column list and output.
2. Write the functional dependencies and find a candidate key.
3. Point to repeated province values in the report and use them to explain the update, insert and delete anomalies. Do not change the source tables.
4. Split the report into province, district and commune tables with their keys.
5. Join your three tables back together and compare the result with the original report: the same number of rows and the same values.

Homework

1. In your own words, explain 1NF, 2NF, 3NF and BCNF using this dataset.

2. Explain when a flat report like this one is useful, and how you would refresh it when the source tables change.

Capstone sprint

C8 / Checkpoint 2: Submit normalized schema v1 with the ERD, functional dependencies, migration SQL and import script. Explain any table you chose not to normalize.

Submit

Write your answers in `work.sql` and label each one, for example `-- Lab 3`. Answer the questions in `reflection.md` and save your query output in `results/`. Then commit and push to your repository. See SUBMISSIONS.md for details.